Written Report

A. Project Overview

- Goal: What questions are you answering?
    - Initiative: How can we train model base on past patient's data and give recommended surgery and survival months for clinical use.
    - Base on clinical features from past patients, what surgery type should be predicted and recommended if we have a new patient? After conducting the surgery, how long the patient can survive?
- Dataset: Source, size (link if too large for GitHub).
    - Kaggle, 8.39 MB, with 1000+ patients' data https://www.kaggle.com/datasets/raghadalharbi/breast-cancer-gene-expression-profiles-metabric/data

B. Data Processing

- How you loaded it into Rust.
  Since this file contained 639 columns and I only need to use part of it, I manually delete 600+ columns that related to genes, which are not related to my questions. Then, I upload it into Rust in to the project folder "data". But still, there are 1000+ patients' data with 30 clinical features.
- Any cleaning or transformations applied.
  I dropped the columns with missing data percentage exceeds 20%, and I add log-transformation for tumor size and nodes of positive lymph nodes.

C. Code Structure

1. Modules
    - model.rs: use training data and apply to Random Forest model for predicting suitable surgery type
    - regression.rs: trains a decision tree regressor for predicting survival months after having the recommended surgery (I used a linear regression before, but due to low performance, Professor Gardos recommended me to use a Decision Tree to improve accuracy, so I used ChatGPT and online resources like Geeksforgeeks to edit this part)
    - data.rs: load patient data and prepare numerical and categorical labels for training models.
    - main.rs: load data, split data for training and testing, evaluate the accuracy, and provide interactive prediction for a new patient.
2. Key Functions & Types (Structs, Enums, Traits, etc)

main.rs

1. main()

a. Purpose
   Runs the overall pipeline:
   i. Loads data
   ii. Prepares datasets (classification & regression)
   iii. Trains models
   iv. Evaluates them
   v. Runs interactive prediction **Trait use here: Display and Debug for print data to console**
b. Inputs/Outputs
   i. Input: None
   ii. Output: Result<(), Box<dyn Error>> **Trait use here: Error, for returning any types of error**
c. Core Logic
   i. Calls load_data(), prepare_surgery_dataset(), and prepare_survival_dataset().
   ii. Splits data into train/test sets using train_test_split().
   iii. Trains: RandomForest classifier and DecisionTree regressor
   iv. Evaluates both models and prints metrics.
   v. Invokes interactive_prediction() to allow manual data entry and predictions.

2. Function: interactive_prediction()
   a. Purpose: Interactive loop to collect user data, predict surgery type, and estimate survival months.
   b. InputsOutputs
      - Input: Classifier: RandomForestClassifier<f64>, Regressor: DecisionTreeRegressor<f64>, Feature names: &[String]
      - Output: Result<(), Box<dyn Error>>
   c. Core Logic
      i. Loops through each feature to prompt user input.
      ii. Displays specific help (categorical options/ranges).
      iii. Predicts surgery type and survival months:
         1. Classification: Predicts surgery type using the classifier.
         2. Regression: Predicts log-survival, then exponentiates to get months.

data.rs

1. Type: Record (Struct)
   a. Purpose: A dynamic representation of a CSV row where each column is a key-value pair.
   b. Inputs/Output
      i. Input: One CSV row.
      ii. Output: HashMap<String, Value> (serde_json Value allows numbers, strings, etc.).

c. Core Logic
    i. Make all fields from a CSV row into a single map, **the key Trait here is Deserialize from serde.**

2. Function: load_data()
    a. Purpose: Loads a CSV into a Vec<Record>.
    b. Inputs/Outputs
        i. Input: file_path: &str
        ii. Output: Vec<Record>
    c. Core Logic: Uses csv::ReaderBuilder to deserialize each row into Record.

3. Function: identify_noisy_columns()
    a. Purpose
    Detects columns with a high % of missing data
    b. Inputs/Outputs
        i. Inputs: records: &[Record], missing_threshold: f64
        ii. Output: Vec<String> (column names to drop)
    c. Core Logic
        i. Counts null/empty string values for each field.
        ii. Filters columns where (missing count / total rows) > threshold

4. Function: prepare_surgery_dataset()/Prepare_survival_dataset()
    a. Purpose: Prepares the classification dataset for surgery prediction.
        i. Inputs/Outputs
            1. Input: &[Record]
            2. Output: (DenseMatrix<f64>, Vec<f64>, Vec<String>)
            (features, labels, feature names)
        ii. Core Logic: Filters out noisy columns, encodes categorical fields using encode_string(), and applies transform_value() for log-scaling where needed. And finally collects feature rows and surgery labels.

model.rs

1. Function: train_random_forest_classifier()
    a. Purpose: Trains a RandomForest classifier for surgery type.
    b. Inputs/Outputs
        i. Inputs:
            1. x_train: &DenseMatrix<f64>
            2. y_train: &Vec<f64>
        ii. Output: RandomForestClassifier<f64>

    c. Core Logic: Uses SmartCore's RandomForestClassifier::fit() with: 100 treesa and max depth 10

2. Function: evaluate_classifier()

a. Purpose: Evaluates classifier accuracy on test data.
b. Inputs/Outputs
    - Inputs: Trained model, x_test, y_test
    - Output: f64 (accuracy)

c. Core Logic: Calls model.predict() and computes accuracy via smartcore::metrics::accuracy. **Trait: Predictor**

regression.rs

1. Function: train_tree_regressor()
    a. Purpose: Trains a Decision Tree regressor for predicting survival months
    b. Inputs/Outputs
        i. Inputs: x_train: &DenseMatrix<f64>, y_train: &Vec<f64>
        ii. Output: DecisionTreeRegressor<f64>
    c. Core Logic: Uses SmartCore's DecisionTreeRegressor::fit() with max depth of 10, min_sample_split:2, and min_sample_leaf of 1
2. Function: evaluate_regressor()
    a. Purpose: Computes mean squared error (MSE) for regression.
    b. Inputs/Outputs
        i. Inputs: Trained regressor, x_test, y_test
        ii. Output: f64 (MSE)
    c. Core Logic: Predicts using the regressor and calculates MSE via smartcore::metrics::mean_squared_error.

3. Main Workflow
    - main.rs coordinates the workflow through reading the data from the csv file and drop the noisy columns and irrelevant data. Then, it split the data into training and testing datasets that will facilitate the modeling for classification and regressor. After the modeling and evaluation, the main will facilitate an interactive section for inputting new data from a patient and give recommended surgery and predictive survival month.

D. Tests

- cargo test output (paste logs or provide screenshots).

```
[/opt/app-root/src/final_project]
$ cargo test
   Compiling final_project v0.1.0 (/opt/app-root/src/final_project)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 1.03s
     Running unittests src/main.rs (target/debug/deps/final_project-b7c4e59ad1767c07)

running 4 tests
test tests::test_data_loading ... ok
test tests::test_prepare_surgery_dataset ... ok
test tests::test_prepare_survival_dataset ... ok
test tests::test_model_training_and_prediction ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 1.73s
```

- For each test: what it checks and why it matters.
    - test_data_loading
        - What it checks: Confirms that the CSV data file is successfully loaded and that the resulting dataset is not empty.
        - Why it matters: Ensures that the data source is accessible and readable
    - test_prepare_surgery_dataset
        - What it checks: Verifies that the surgery dataset is properly prepared through checking the number of feature rows matches the number of labels
        - Why it matters: Ensure the dataset is suitable for classification training.
    - test_prepare_survival_dataset
        - What it checks: Similar to the previous test but for the survival dataset (regression).
        - Why it matters: Confirms that the regression dataset is prepared properly, and survival prediction depends on this structure.
    - test_model_training_and_prediction
        - What it checks: it checks both the Random Forest Classifier and Decision Tree Regressor, evaluates them to make sure the accuracy is between 0 to 1
        - Why it matters: ensure the workflow is workable

E. Results

- All program outputs (screenshots or pasted).

  Surgery Dataset:
  Records: 1799
  Dropped noisy columns: ["tumor_stage"]
  Features per record: 17

  Survival Dataset:
  Records: 1799

Dropped noisy columns: ["tumor_stage"]
Features per record: 18

Datasets Loaded:
Surgery dataset: 1799 samples, 17 features
Survival dataset: 1799 samples, 18 features

Evaluation Results:
Surgery Accuracy: 0.7922
Survival MSE: 1.21
New Patient Input
Answer for: age_at_diagnosis
Typical Range: 20-90 years old
Input for age_at_diagnosis: 45
Answer for: chemotherapy
Options: 0 = No, 1 = Yes
Input for chemotherapy: 1
Answer for: neoplasm_histologic_grade
Options: 1, 2, 3
Input for neoplasm_histologic_grade: 1
Answer for: hormone_therapy
Options: 0 = No, 1 = Yes
Input for hormone_therapy: 1
Answer for: lymph_nodes_examined_positive
Typical Range: 0-45 nodes
Input for lymph_nodes_examined_positive: 12
Answer for: nottingham_prognostic_index
Typical Range: 1.0 - 6.0
Input for nottingham_prognostic_index: 1
Answer for: radio_therapy
Options: 0 = No, 1 = Yes
Input for radio_therapy: 1
Answer for: tumor_size
Typical Range: 5mm - 100mm
Input for tumor_size: 20
Answer for: cancer_type_detailed
Options: 0 = Ductal, 1 = Lobular, 2 = Mixed
Input for cancer_type_detailed:
Answer for: cellularity
Options: 0 = Low, 1 = Moderate, 2 = High

Input for cellularity: 0
Answer for: pam50_subtype
Options: 0 = LumA, 1 = LumB, 2 = HER2, 3 = Basal, 4 = Normal, 5 = Claudin-low
Input for pam50_subtype:
Answer for: er_status
Options: 0 = No, 1 = Yes
Input for er_status: 1
Answer for: her2_status
Options: 0 = No, 1 = Yes
Input for her2_status: 1
Answer for: inferred_menopausal_state
Options: 0 = Pre, 1 = Post
Input for inferred_menopausal_state: 0
Answer for: integrative_cluster
Options: 1-10
Input for integrative_cluster: 3
Answer for: primary_tumor_laterality
Options: 0 = Right, 1 = Left, 2 = Other
Input for primary_tumor_laterality:
Answer for: pr_status
Options: 0 = No, 1 = Yes
Input for pr_status: 1

Recommended Surgery Type: Mastectomy
Which surgery did the patient actually receive? (0 = Breast Conserving, 1 = Mastectomy)
1

Predicted Overall Survival: 170.1 months

- Interpretation in project context (no need for "groundbreaking" results).
  The main goal of the project is to provide a clinical tool for faster and relatively accurate recommendations for the patient's recommended surgery type and the predicted survival month length after the surgery. The Random Forest Model now is at least 75% accuracy in terms of matching the surgery type, and then, the MSE for survival month after using the regressor is around 2 months. Generally, the performance of the model and regressor is okay, but it is not applicable in reality. The limitation is due to the the missing 600 features from the genetic features, which are not included in this project because taking them into consideration takes more coding. Meanwhile, even though the code determines that the tumor stage is considered as the noisy column, it provides valuable insight into the month of living and the recommended surgery type. There should be more advanced

and proper classification models, and it would be better if I could use random forest regressor for predicting survival time. But since this course did not get a change to introduce it, hope I can advance it in the future.

F. Usage Instructions

- How to build and run your code.
  Use cargo run and follow the instructions step by step to input the data for interaction.
- Description of any command-line arguments or user interaction in the terminal.
  Just follow ths instruction to put input. Everything will be given. If feel unsure, just use "return" to skip the question.
- Include expected runtime especially if your project takes a long time to run.
  Within 1 min for both cargo run or cargo run –release.

G. AI-Assistance Disclosure and Other Citations

- Cite any substantive ChatGPT/GenAI you used (e.g. screenshot or description).
  - When loading data and identifying noisy columns, I had trouble with identifying empty cell and data for null and empty strings. Thus, I asked chatGPT to filter out

the columns base on missing data percentage threshold.

give a debugged code on identify_noisy_columns filter out the columns base on missing data percentage threshold.

Awesome request! ✅ Let's **focus on debugging and refining** your existing `identify_noisy_columns` function to **ensure it reliably filters out noisy columns** based on missing data percentage.

🛠️ Problem:

The original code you posted works conceptually but might miss edge cases, e.g.:

- **Empty strings vs nulls**
- **Case where all rows have no columns** (rare but possible)
- **Ensure we treat `"NA"` and similar markers as missing too**

Also, I will **add debug-friendly output** so you can easily **see which columns are being checked and their missing rates.**

✅ Here's a **debugged + improved version:**

rust Copy Edit

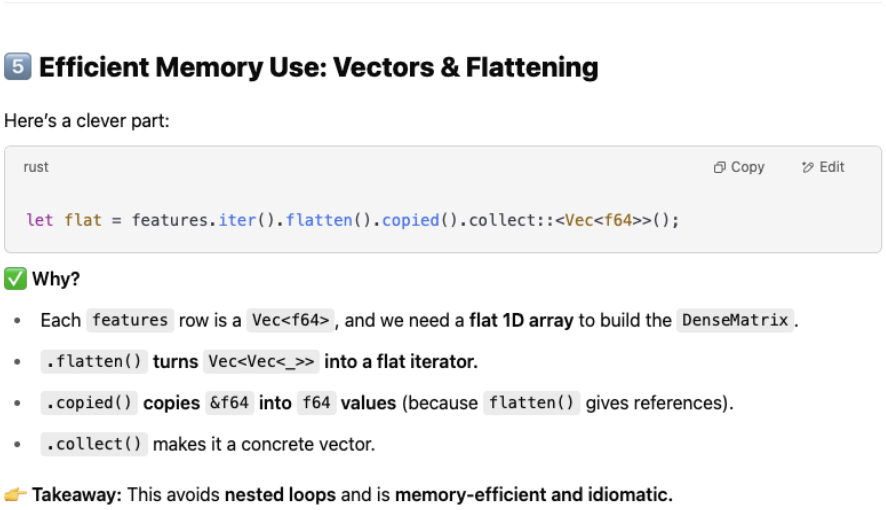
```rust
/// Identify noisy columns (columns with too much missing data)
pub fn identify_noisy_columns(records: &[Record], missing_threshold: f64) -> Vec<String>
    if records.is_empty() {
        println!("No records provided.");
        return vec![];
    }

    let total = records.len() as f64;
    let mut missing_counts: HashMap<String, usize> = HashMap::new();

    // Loop through each record and each field to count missing data
    for record in records {
        for (key, val) in &record.fields {
            let is_missing = val.is_null()
                || matches!(val, Value::String(s) if s.trim().is_empty() || s.trim().eq_i
                || matches!(val, Value::Number(n) if n.as_f64().unwrap_or(0.0).is_nan());

            if is_missing {
                *missing_counts.entry(key.to_string()).or_insert(0) += 1;
            }
        }
    }
```

- I tried to ask ChatGPT for better memory use.



5️⃣ Efficient Memory Use: Vectors & Flattening

Here's a clever part:

rust Copy Edit

```rust
let flat = features.iter().flatten().copied().collect::<Vec<f64>>();
```

✅ **Why?**

- Each `features` row is a `Vec<f64>`, and we need a **flat 1D array** to build the `DenseMatrix`.
- `.flatten()` **turns** `Vec<Vec<_>>` **into a flat iterator.**
- `.copied()` **copies** `&f64` **into** `f64` **values** (because `flatten()` gives references).
- `.collect()` makes it a concrete vector.

👉 **Takeaway:** This avoids **nested loops** and is **memory-efficient and idiomatic.**

  - You can skip this for common knowledge/debugging use cases

- You can also provide links to other sources you found useful that are not "common knowledge"
  - https://www.geeksforgeeks.org/random-forest-algorithm-in-machine-learning/

Code Comments (I keep the record here for my personal reference, they are also in the code, and I use this for writing key functions in the written report)

- Module Header
    - One-sentence summary of the module's purpose for each module that you have.
        - model.rs: use training data and apply to Random Forest model for predicting suitable surgery type
        - regression.rs: trains a decision tree regressor for predicting survival months after having the recommended surgery (I used a linear regression before, but due to low performance, Professor Gardos recommended me to use a Decision Tree to improve accuracy, so I used ChatGPT and online resources like Geeksforgeeks to edit this part)
        - main.rs: load data, split data for training and testing, evaluate the accuracy, and provide interactive prediction for a new patient.
        - data.rs: load patient data and prepare numerical and textual labels for training models
- Function Comments
    - model.rs: use training data and apply to Random Forest model for predicting suitable surgery type
        - What it does:
            - fn Train_random_forest_classifier: Trains a random forest using training data
            - fn evaluate_classifier: Uses classification accuracy to measure the model's accuracy for prediction
        - Inputs and outputs
            - fn Train_random_forest_classifier:
                - Input:
                    - Featurest: x_train: DenseMatrix<f64>
                    - Labels: y_train: Vec<f64>
                - Output: a trained randomforest model
            - fn evaluate_classifier:
                - Trained model: model: RandomForestClassifier<f64>
                - Features: x_test: DenseMatrix<f64>
                - Labels: y_test: Vec<f64>
        - High-level logic (key loops, iterators, algorithms)
            - fn Train_random_forest_classifier:
                - Call RandomForestClassifier::fit(...) using the Smartcore library to construct the model

            - **Algorithm:** Randam forest here is an ensemble of decision tress where each tree is trained on a random subset of the data and feature, finally the tree's decision will turn into a final prediction
        - fn evaluate_classifier:
            - Test the predicted result with the actual result to get accuracy of the model
    - Fundamental functions: presented above.
- data.rs
    - What it does: load the data, conduct data transformation, prepare data for machine learning models
        - Suregry recommendation (Classification)
        - Survival month prediction (Regression)
    - Inputs and outputs:
        - fn load_data:
            - Input: CSV file's data
            - Output:
                - Data for Surgery recommendation: a matrix of features, feature name, and surgery label
                - Data for Survival month prediction: a matrix of features, label of survival months, and feature names
        - fn identify_noisy columns:
            - Input:dataset, which is a Vec, and threshold of the percentage of missing data
            - Output: Vec of column names that need to be dropped
        - fn prepare_surgery_dataset & prepare_survival_dataset
            - Input: dataset
            - Output: a tuple that consists of features, labels, and feature names
    - High-Level Logic
        - After loading the csv file, I need to separate the data into two parts for RandomForest model training and regression.
        - Thus, I need to clean the data first through identifying and dropping the noisy columns with too many missing and unuseful data.
        - Since the dataset contains categorical data, I need to encode the categorical variable using numerical encoding for strings.
        - For normalization, I apply transformations for particular features.
        - Finally, I create the dataset for classification and regression.

        - Fundamental functions:
            - Transform_value: use log transformation for tumor size and lymph nodes for normalization so that the model can have better accuracy and performance
            - Selected_features: hardcoding the list of features
    - regression.rs
        - What it does: trains and evaluates a Decision Tree Regressor for predicting survival months after having surgery
        - Inputs and Outputs
            - fn train_tree_regressor: trains the decision tree using training data
                - Input:
                    - X_train: DenseMatrix<f64> - the training features (the clinical featrues)
                    - Y_train: a Vec<f64> - the training labels, which is the survival month
                - Output: a regression model DecisionTreeRegressor<f64>
            - fn evaluate_regressor: use regression to give the predicted survival month lengths
                - Input:
                    - Model: the trained model
                    - X_test: test features
                    - Y_test: test label
                - Output: Mean Squared Error for predictions
        - High-level logic
            - The decision tree regressor can help to handle mixed data types since the clinical features contain both numerical and categorical variables. Meanwhile, the relationship is not linear, so it is unsuitable to use a linear regressor.
        - Fundamental functions:
            - N/A
    - main.rs
        - What it does:controls the overall workflow through calling the functions to train the random forest model, decision tree regressor, and give an interactive function for inputting the patient's features to give out the recommended surgery type and the survival time after the surgery. And it includes the tests for correct data loading, feature cleaning, and model training.
        - Inputs and Outputs
            - fn main: the entry point of the program, and the output is a return type which means that it can be Ok() or fail with error through

error handling in this whole program using Box<dyn std::error::Error>>

- fn interactive_prediction: collect a new patient's data from the user
  - Input:
    - classifer: the trained RandomForest model for surgery
    - Regressor: the trained regression for survival prediction
    - Feature_names: the feature names used to build the prediction input
  - Output:
    - Ok() and Box<dyn std::error::Error>>

- High-level logic:
  - Honestly this part is very hardcoded. The inputs values are collected one byone, and the missing data will be treated as default. Two densematric are built from user inputs, where the first one is for surgery prediction and the other is for survival prediction with a new added input Surgery Type.
- Fundamental functions: N/A

- In-Function Annotations: At each "crucial" step (e.g. data transformations, core calculations, complicated use of iterators that cannot be easily read, etc), briefly explain why it's there or what it does.
  - model.rs
    - Imports: use Smartcore to create the machine learning model, and import accuracy metric to evaluate the predictions, and lastly import Dense Matrix to store feature data because each row represents a patient, and each column represents a clinical feature

```
use smartcore::ensemble::random_forest_classifier::{RandomForestClassifier, RandomForestClassifierParameters};
use smartcore::metrics::accuracy;
use smartcore::linalg::naive::dense_matrix::DenseMatrix;
```

- In fn train_random_forest_classifer, use method .fit() to train the model where I design 100 decision trees with a max depth of 10. For error handling, I add .expect() if training fails.
- In fn evaluate_classifer, I input the model and testing data and output the accuracy. Again, I use .expect() if the accuracy score fails to return.
  - Accuracy = correct prediction / total sample

  - data.rs

- The use of imports here mainly aim to work with CSV files and try to read the file effectively using BufReader, and the HashMap is used to hold row data.
- To avoid hardcoding too much, I use #[serde(flatten)] after creating a Struct to present the datasetrecord so that the columns names can be allowed here. And within the process of reading CSV file, I deserialize the rows into he record and collect all rows into a vector. The error handling here is using .expect when it fails.
- After loading the CSV file, I need to identify and drop the noisy columns. In doing so, I use

  ```
  if val.is_null() || matches!(val, Value::String(s) if
  s.trim().is_empty())
  ```

  to treat both empty data (.is_null()) and empty string, and I count the missing data for each columns and keep columns were missing precentage is above the threshold. I create a HashMap where the key is the column's name and the value is the count of missing values. I use for loop to iterate the column-value pair and increment the missing count using if condition. The inner loop is the columns in one record, and the out loop is for all records. The threshold I put 0.2.
- The dataset contains mixed datatypes, so I need to encode string into numbers. So for all the categorical data, I match the position data options into numerical inputs such as

  ```
  "cellularity" => match val.to_lowercase().as_str() {
     "low" => 0.0, "moderate" => 1.0, "high" => 2.0
  ```

  So I can use them for further modeling.

- The transformation part is easy because I simply tried different transformation on the numerical features, and I found tumor size and numbers of lymph nodes can use log transformation to improve the model performance. Thus, I just used the log function (value + 1.0).ln().
- Selected_feature is a helper function that returns a vectors of strings including all the features, I use &'static to ensure it lives throughout the prorgam because they are the most important names for making sure the program can compile.
- The prepare_surgery_dataset and prepare_survival_dataset are basically in the same logic. I use &[Record] to borrow the data so that the function can

only read the data, and I use pattern matching for value transforming and string endoing. Then for useful features, I use .into_inter() to convert vectors into interators, .filter() to keep valid features, .map() to turn string slice into String for ownership, and bring the iterators back to Vec usin vec().
        - Because my program toke a huge amount of memory and the compiling time was super long, so I asked ChatGPT to advance my code. I use flat here.
        - The only difference between the two dataset is features, The survival month needs the “overall_survival_month” column.
    - regression.rs
        - The imports here are basically for smartcore machine learning.
        - I set the max depth of 10, minimum sample split of 2, and minimum sample leaf of 1. This part is mainly a modification of the linear regression.
    - main.rs
        - The key part in main.rs is to split the dataset into testing and training (20/80), anything else is just calling functions from the other modules.
        - In the interaction function, there is a loop until all the valid inputs, and to make sure the patients have missing input, I assign the default value.
        - Since in the same function but I need to bring in the new input for predicting survival month. I use io::stdin().read_line(&mut surgery_input)?; to read the input and stores to a new string, then I parse the inputand add it into the survival features.
- Structs & Enums
    - The only sturct use in this project is Record, it irepresents a single row of the CSV dataset. It is a dynamic container with flexible fields that can hold mixed data types in the dataset.